

รายงานการออกแบบ อัลกอริทึม และผลการทดลองระบบคิว

โครงการ: ระบบจัดการคิวคำสั่งซื้อ (Food Delivery Order Management System)

1. การออกแบบคิว (Queue Design)

ในการออกแบบระบบจัดการคิวอาหาร การเลือกโครงสร้างข้อมูลมีความสำคัญต่อทั้งประสิทธิภาพเชิงเวลาและความจุหน่วยความจำ:

- การเลือกใช้โครงสร้างข้อมูล (Implementation Choice):**
 - Algorithm A (FIFO Queue):** เลือกใช้ ArrayDeque เนื่องจากให้ประสิทธิภาพความเร็วในการทำงานสูง $O(1)$ ทั้งการ Enqueue (offer) และ Dequeue (poll) โดยไม่มี Overhead ในการเรียงลำดับข้อมูล
 - Algorithm B (Priority Queue):** เลือกใช้ PriorityQueue ซึ่งใช้โครงสร้างภายในแบบ Binary Min-Heap บน Array ในการจัดลำดับความสำคัญตามเงื่อนไข Priority (1 = Express, 2 = Normal, 3 = Large) หาก Priority เท่ากันจะพิจารณาจาก orderTime ตามลำดับ
- การบริหารจัดการหน่วยความจำ (Space Allocation):**
 - ใช้ประเภท **Unbounded Queue** บน JVM Heap Memory ซึ่งรองรับข้อมูลได้สูงสุดตามขนาด N วัตถุ โดยมี Space Complexity ในการจัดเก็บข้อมูลเท่ากับ $O(N)$

2. ความถูกต้องของอัลกอริทึมและสัจพจน์ (Correctness & Invariant)

การการันตีว่าระบบประมวลผลได้อย่างถูกต้อง 100% โดยไม่เกิดข้อผิดพลาด ใช้หลักการพิสูจน์ดังนี้:

- Queue Correctness Axioms:**
 - FIFO Correctness:** หาก A ถูก Enqueue เข้ามาก่อน B แล้ว A จะต้องถูก Dequeue ออกมาก่อน B เสมอ (กรณี Algorithm A)
 - Priority Correctness:** หาก $Priority(A) < Priority(B)$ แล้ว A จะต้องถูก Dequeue ออกมาก่อน B เสมอ (กรณี Algorithm B)

- **Conservation of Items Invariant:** ณ เวลาใดๆ จำนวน Order รวมทั้งหมดจะต้องเท่ากันเสมอ:

$$\text{Processed Orders} + \text{Remaining Queue Size} = N_{\text{total}}$$

- **Loop Invariant** ในฟังก์ชัน `processQueue` :

“ณ จุดเริ่มต้นของการลูปในแต่ละรอบ ก่อนการ Dequeue ค่าของ `currentTime` จะเท่ากับผลรวมของ `prepTime` ของ Order ทั้งหมดที่ถูกประมวลผลเสร็จสิ้นไปแล้ว”

- **Initialization:** ก่อนเริ่มลูปแรก ยังไม่มี Order ถูกทำ `currentTime = 0` → เป็นจริง
- **Maintenance:** ในแต่ละรอบ ลูปนำ Order O_k ออกมาและอัปเดต `currentTime += O_k.prepTime` สำหรับรอบถัดไป → เป็นจริงเสมอ
- **Termination:** เมื่อคิวว่าง (`queue.isEmpty()`) หมายถึงทุก Order ถูกประมวลผลครบถ้วน และค่าเวลารอคอยเฉลี่ยคำนวณได้อย่างถูกต้องแม่นยำ

3. ผลการเปรียบเทียบเชิงทดลอง (Experimental Comparison)

3.1 การเปรียบเทียบเวลารอคอย (เชิงคุณภาพ - Scenario $N = 5$)

ตัวแปรวัดผล

Algorithm A (FIFO Queue)

Algorithm B (Priority Queue)

ผลลัพธ์เชิงเปรียบเทียบ

ลำดับการประมวลผล

O1 → O2 → O3 → O4 → O5

O2 → O4 → O1 → O3 → O5

Algorithm B ลัดคิวให้ Express (O2, O4) ก่อน

Average Waiting Time

20.60 นาที

16.80 นาที

Algorithm B เร็วกว่า ~18.4%

Maximum Waiting Time

38.00 นาที (เกิดขึ้นที่ O5)

38.00 นาที (เกิดขึ้นที่ O5)

เท่ากัน (ออเดอร์ Large อยู่ท้ายคิวทั้งคู่)

3.2 การเปรียบเทียบระยะเวลาประมวลผลโปรแกรม (เชิงประสิทธิภาพ - Execution Time)

ขนาดข้อมูล (N)

FIFO Queue (ArrayDeque)

Priority Queue (PriorityQueue)

การวิเคราะห์

$N = 100$

$< 1 \text{ ms}$

$< 1 \text{ ms}$

ประสิทธิภาพใกล้เคียงกัน

$N = 1,000$

ประมวลผลได้รวดเร็ว

ช้ากว่า FIFO เล็กน้อย

เริ่มมี Overhead จากการ Heapify

$N = 10,000$

รักษาระดับความเร็วได้ดี

ช้ากว่า FIFO ชัดเจน

ผลกระทบจาก Complexity $O(N \log N)$

$$N = 50,000$$

ประหยัดเวลาสูงสุด ($O(N)$)

ใช้เวลานานที่สุด ($O(N \log N)$)

Heap Rebalance ทำงานหนักตามขนาด N

4. สรุปผลและการแลกเปลี่ยน (Trade-offs Summary)

1. ด้านเวลารอคอย (Waiting Time): **Priority Queue** ชนอย่างชัดเจน ช่วยลดเวลารอเฉลี่ยของลูกค้าได้ แต่มีข้อควรระวังคืออาจเกิดปัญหา **Starvation** กับออเดอร์ประเภท Normal/Large หากมี Express เข้ามาต่อเนื่อง
2. ด้านประสิทธิภาพโปรแกรม (Execution Time): **FIFO Queue** ชนอย่างชัดเจน มี Time Complexity การทำงานเป็น $O(1)$ ต่อ Operation ทำให้เหมาะกับระบบที่มี Transaction ใหญ่มาก (N สูง)